Operations & Advanced Features PRD

Admin console, real-time updates, events, analytics, simulation, security and testing

1. Operations Console

The admin application is the operational control surface for QuickFlow. It should expose the state of the system in a dense, precise format rather than mimic the customer storefront.

2. Admin Navigation

```
Overview Orders Fulfillment Tasks Warehouses Workers Inventory Scheduling Simulation Analytics System Events Settings
```

3. Dashboard Requirements

- Show order volume, active tasks and late orders.
- Show warehouse utilization and operational status.
- Show available, busy and offline workers.
- Show pending, processing, completed and failed tasks.
- Provide direct access to scheduling and simulation.

4. Order Decision View

Opening an order should show its fulfillment pipeline plus the reasoning behind the warehouse decision.

Decision Evidence	Example
Inventory	Full inventory available
Capacity	82% worker capacity
Processing	Estimated 18 minutes
Score	Lowest fulfillment score

This makes the system explainable and gives managers a practical way to audit automated decisions.

5. Task Queue

The admin can view the current task order and the active scheduling strategy. Switching between FIFO, Priority, Deadline and Hybrid should cause the queue to be recalculated by the engine rather than merely changing a visual label.

Operations & Advanced Features PRD

Admin console, real-time updates, events, analytics, simulation, security and testing

6. Warehouse & Inventory Views

Warehouse views should expose inventory, worker capacity, active orders, task load and operational status. A network visualization can represent relationships between warehouses and delivery locations.

Inventory should show stock by warehouse, reserved quantity, available quantity and recent stock movement. Reservation history should connect stock changes back to orders.

7. Worker View

Workers should expose status, department, current task, completed tasks, average task time and success rate. This helps explain why a task was assigned or why a warehouse may be nearing capacity.

8. Real-Time Updates

```
C++ Engine → Node Backend → WebSocket / Socket.IO → React
```

Order, task and worker state changes should propagate without requiring the admin to refresh the page. Customer tracking can use the same event/state stream, filtered to customer-safe information.

9. Event Logging

Important actions should produce events such as OrderCreated, InventoryReserved, WarehouseAssigned, TaskCreated, TaskAssigned, TaskCompleted, OrderPacked, QCCompleted and OrderShipped.

An event timeline should provide an auditable history of an order and the operational decisions made along its journey.

10. Analytics

Metric	Purpose
Average processing time	Measure order/task execution speed.
Average fulfillment time	Measure end-to-end fulfillment performance.
Warehouse utilization	Identify overloaded or underused facilities.
Worker utilization	Understand workforce capacity.
Late orders	Measure deadline performance.
Failed tasks	Measure reliability.
Retry rate	Measure operational instability.

Operations & Advanced Features PRD

Admin console, real-time updates, events, analytics, simulation, security and testing

11. Simulation Engine

Simulation is a planning feature, not just a visualization. The manager provides a workload scenario and QuickFlow estimates operational outcomes.

Input	Example
Expected orders	50,000
Workers	30
Warehouses	4
Scheduler	Hybrid

Outputs should include estimated completion time, late orders, worker utilization and warehouse utilization. The engine can also compare scheduling strategies under the same workload.

12. Security

Control	Requirement
Authentication	JWT-based login.
Password security	Password hashing.
Authorization	Role-based access.
Roles	CUSTOMER, WAREHOUSE_MANAGER, ADMIN.

Customers should only access their own orders. Administrative roles may access operational information according to their permissions.

13. Testing Strategy

- **C++ unit tests:** warehouse selection, priority calculation, inventory reservation, scheduling, task generation and state transitions.
- **Backend tests:** order creation and order retrieval APIs.
- **Frontend tests:** order creation, order display, status updates and dashboard behavior.
- **Concurrency tests:** simultaneous reservations for the same limited item.
- **Integration tests:** customer order → engine → database → real-time UI update.

14. Operational Quality Principle

Every important state change should be observable, recoverable where appropriate, and traceable through event history. The system should favor correctness of shared state over optimistic throughput.